

Numerical Methods
EN 530.766
Final Project

Weixuan Li

1. Objective

Consider the 2-D Laplace equation

$$u_{xx} + u_{yy} = 0$$

on the domain (shown in Figure 1) with outer boundary $u=0$ and inner boundary with $u = 1$.

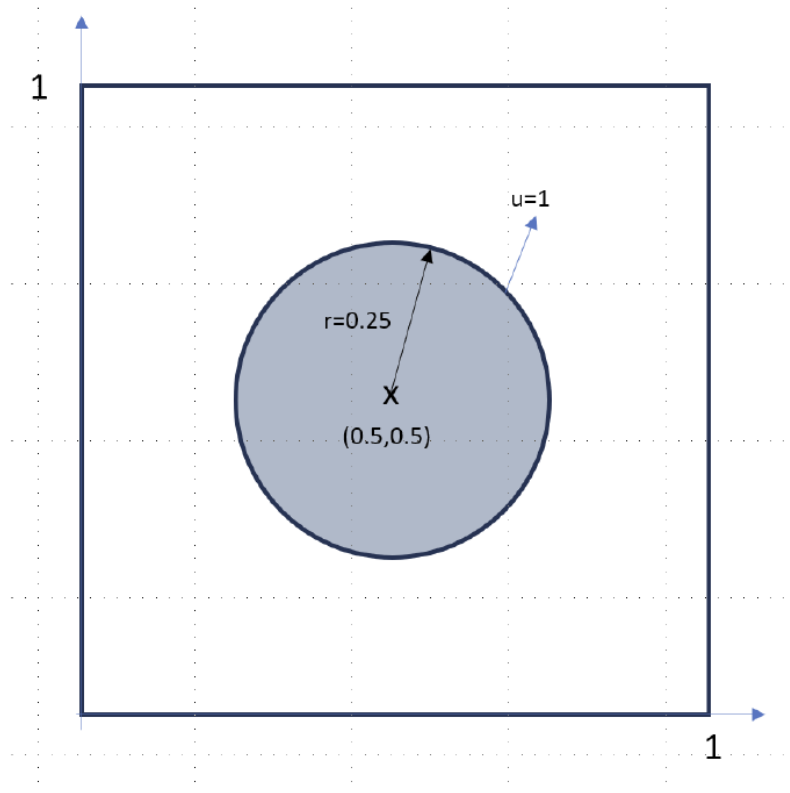


Figure 1 Computation domain of Laplace equation with outer boundary $u = 0$ and inner boundary $u = 1$

2. Methodology

Implement the second-order central difference scheme on the second-order derivatives of u in the equation, to develop the point SOR Gauss-Seidel scheme to solve the problem with inner and outer boundaries. Define N as the number of grids at each edge, and ω as the relaxation parameter of SOR. The flowchart of the point SOR

Gauss-Seidel solver is shown in Figure 2.

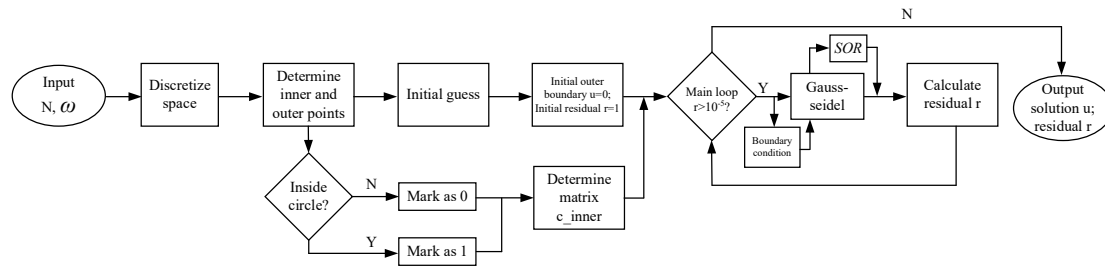


Figure 2 Flowchart of the point SOR Gauss-Seidel solver

3. Results

3.1 Iteration Process

Implement the solver to the problem on grids with 32×32 , 96×96 , 160×160 and 224×224 grid cells. The residuals are calculated as $r^k = \left(\frac{\delta_x^2}{\Delta x^2} + \frac{\delta_y^2}{\Delta y^2} \right)$, and the relaxation parameter is $\omega = 1.0$. With the initial guess of each point in the range of $[-1, 1]$, the total residuals versus iteration index over the convergence process of the point SOR Gauss-Seidel scheme with different resolutions are shown in Figure 3.

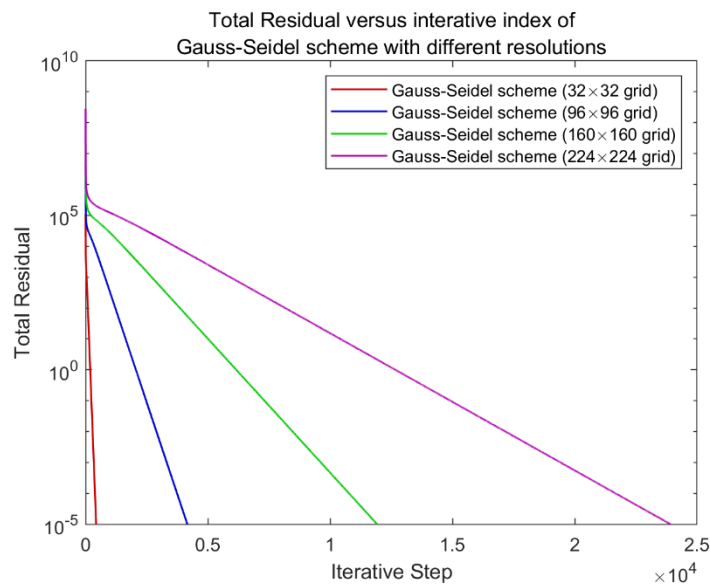


Figure 3 The total residual versus iteration index over the convergence process of the point SOR Gauss-Seidel scheme with different resolutions

The iterative steps needed for each resolution to converge are shown in Table 1.

Grid points	32×32	96×96	160×160	224×224
Convergence step	435	4168	11912	23937

Table 1 The iterative steps needed for each resolution to converge

It can be observed from Figure 1 and Table 1 that the residuals decrease quickly through the iterative process, and eventually reach the convergent state where $r_{total} < 1 \times 10^{-5}$. The rate of convergence varies with different grid resolutions. Higher resolution grids need more steps to converge. Specifically, the 224×224 grids case needs most iterative steps (more than 20,000) to converge, while the 32×32 grids case converges within only hundreds of iterative steps.

3.2 Contour of Converged Solutions

Also, the contour plots of the converged solutions for each resolution are shown in Figure 4. In this case, I assume that the circle in the middle has a constant value of 1 because of the inner boundary condition.

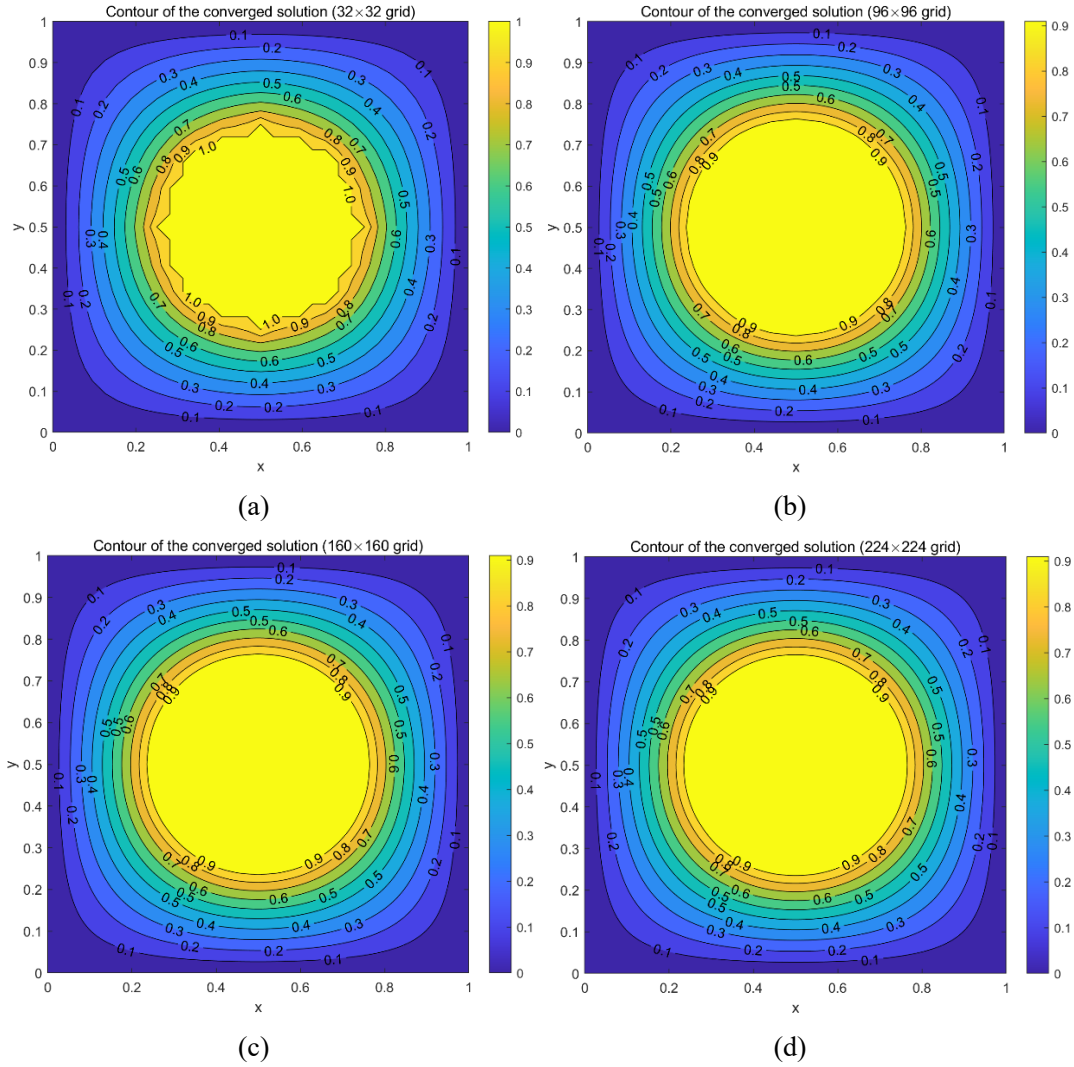


Figure 4 The contours of the converged solutions with different resolution grids: (a) 32×32 grid, (b) 96×96 grid, (c) 160×160 grid, (d) 224×224 grid

Figure 4 illustrates that solution u decreases gradually from the inner boundary to the

outer boundary. Additionally, the contours are influenced by the boundaries, transitioning from circular shapes near the inner boundary to square shapes toward the outer boundary. These results align with physical intuition. Moreover, the gradient of the contours is steeper near the inner boundary and gentler near the outer boundary. In terms of varying resolutions, higher resolutions yield more refined results. For instance, 32×32 grids case produces a more rudimentary operation of the inner boundary, exhibiting an irregular, stair-shape appearance, whereas 224×224 grids case displays the inner boundary that is closer to a circle.

In short, Figure 3 and Figure 4 demonstrate that using higher resolutions may lead to slower convergence with higher computational costs, yet it offers more detailed and refined results. On the other hand, while using coarser grids may provide less detailed outcomes, the convergence process is faster with lower computational costs.

3.3 Results Verification

To check the correctness of the solution, I used a theoretical case that I could obtain the exact solution by changing the outer boundary condition to generate a manufactured solution, and compare my numerical solutions to the manufactured solution. In detail, the problem with the new outer boundary condition is illustrated in Figure 5.

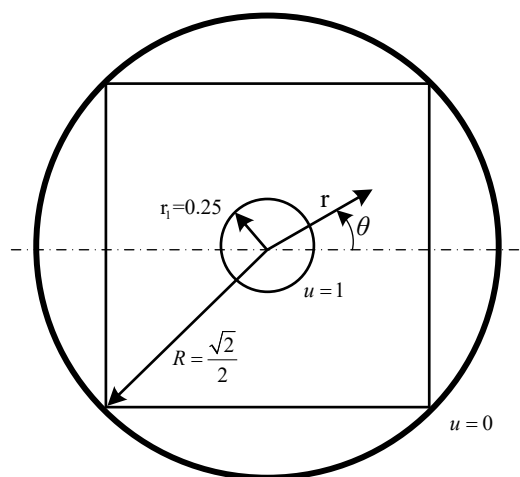


Figure 5 The configuration with modified boundary condition used to generate manufactured solutions

This configuration is radial symmetry, which means the variation of u is not relevant to θ , so the Laplace equation in the polar coordinate for this case is:

$$\frac{\partial}{\partial r} \left(r \frac{\partial u}{\partial r} \right) = 0$$

with the boundary condition:

$$\begin{cases} u(r_1) = 1 \\ u(R) = 0 \end{cases}$$

Then the analytical solution of this equation is:

$$u = \frac{\ln r - \ln R}{\ln r_1 - \ln R}$$

Using the formula of the analytical solution, the exact solutions on the four edges of the square (outer boundary in configuration 1) can be obtained as the new outer boundary condition. Implementing the new boundary condition into our problem, and solve the problem with the Gauss-Seidel solver, I can get a new set of numerical solutions that can be compared to the exact solution. The root mean square errors of the numerical solutions with different grid resolutions are calculated by the formula below, and the values of the error are shown in Table 2.

$$Error = \sqrt{\frac{\sum_{i=1}^{N_{fluid}} (u_{exact} - u_{numerical})^2}{N_{fluid}}}$$

Grid points	32×32	96×96	160×160	224×224
Error	0.0219	0.0082	0.0041	0.0029

Table 2 The root mean square errors of numerical solutions with different grid resolutions compared to the exact solution

It can be noticed that the root mean square errors are minor (<0.03), and the finer the grids, the smaller the errors. This result verifies the correctness of the point SOR Gauss-Seidel solver, and indicates that the higher the resolution used, the more accurate the results the solver can provide.

3.4 Effects of Overrelaxation Parameter

For the point SOR Gauss-Seidel scheme, if we increase the overrelaxation parameter ω , the convergence can be sped up correspondingly. The convergence curves with different overrelaxation parameters for each grid resolution are shown in Figure 6.

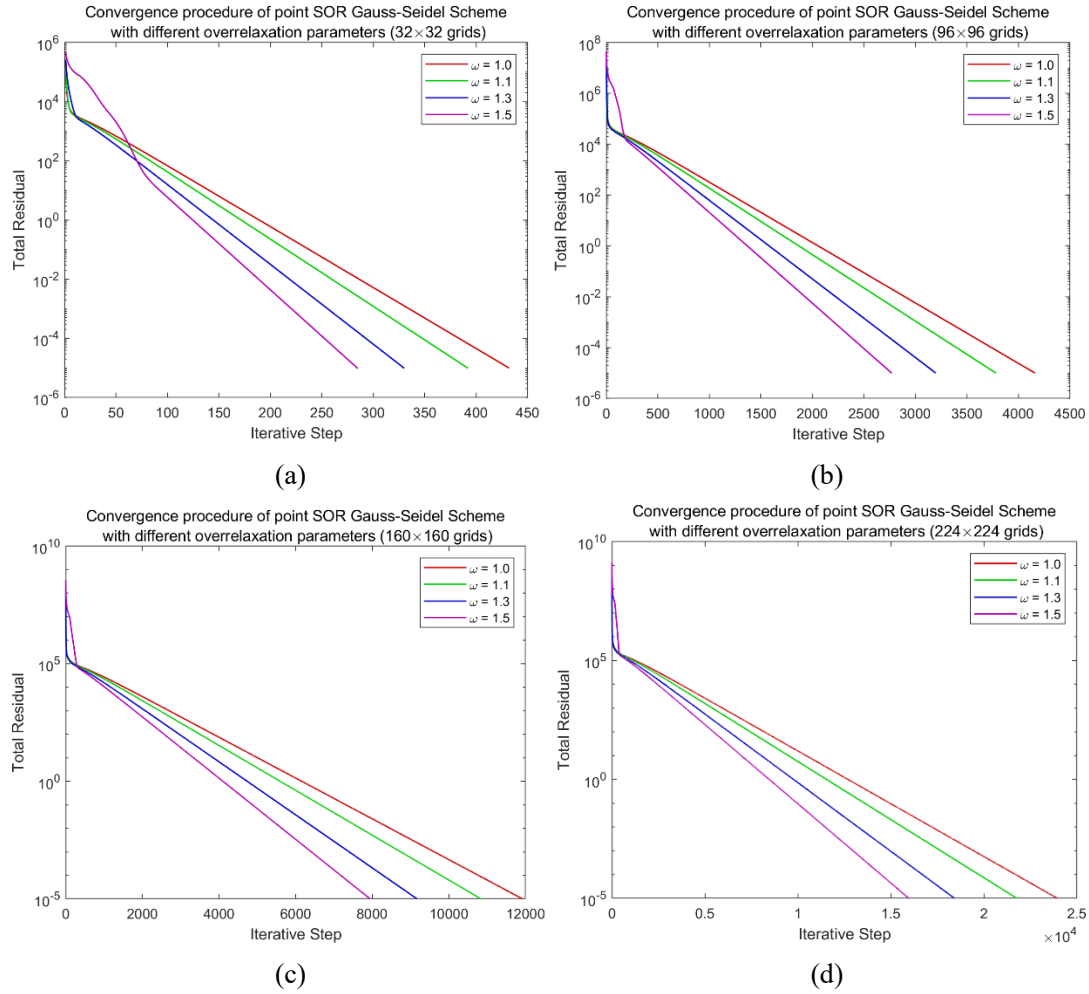


Figure 6 The convergence curves with different overrelaxation parameters for each grid resolution: (a) 32×32 grid, (b) 96×96 grid, (c) 160×160 grid, (d) 224×224 grid

For each grid resolution, the convergence of the point SOR Gauss-Seidel scheme speeds up after being over-relaxed, and the larger the overrelaxation parameter, the faster the convergence. It also needs to be considered that the relaxation parameter cannot be too large since that may cause divergence. We can also find the optimal values of the relaxation parameter for these conditions. To do this, we plot the iterative steps needed to converge of the scheme versus different relaxation parameters for each grid resolution, which is shown in Figure 7, and find the minimal values along the curves in these plots.

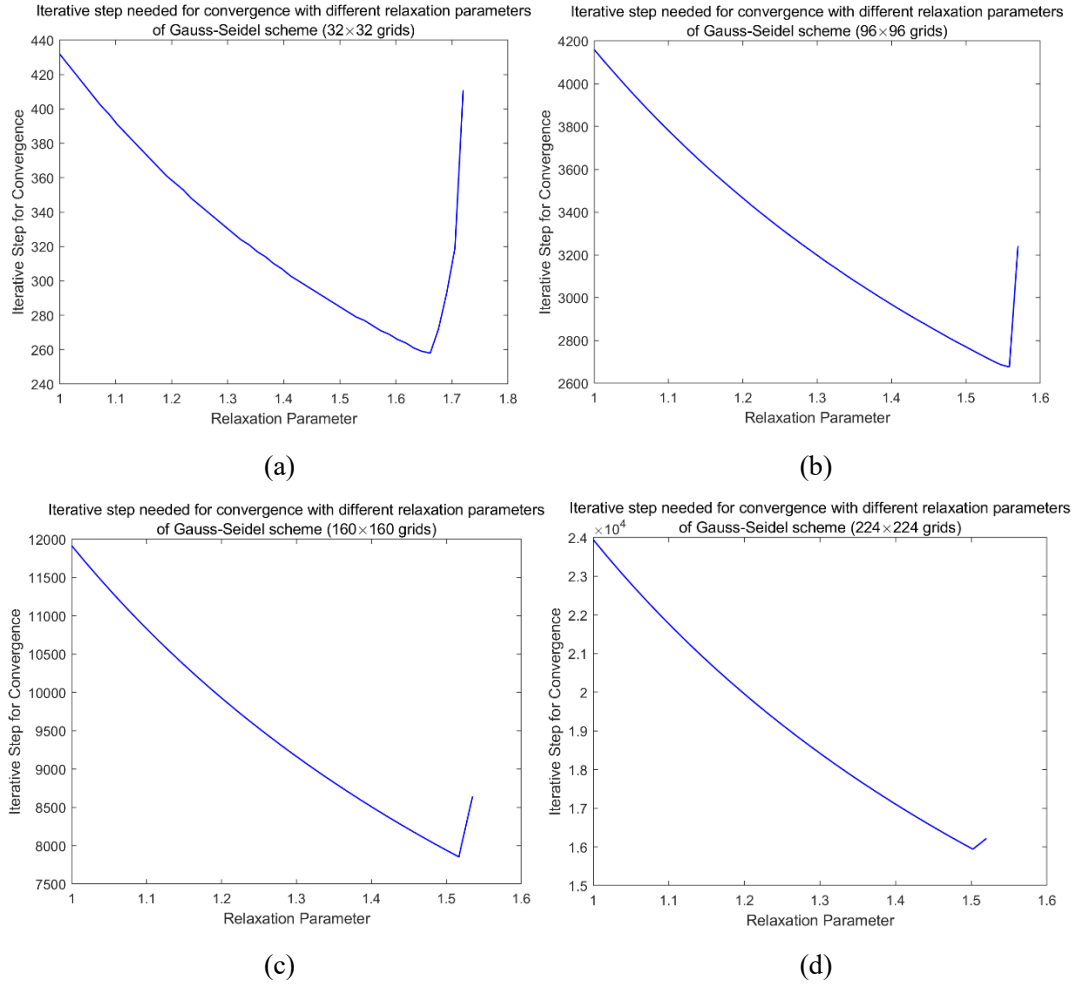


Figure 7 The iterative steps needed to converge with different relaxation parameters of Gauss-Seidel scheme for each grid resolution: (a) 32×32 grid, (b) 96×96 grid, (c) 160×160 grid, (d) 224×224 grid

Find the minimal values of the iterative step of these curves, and then the optimal values of the overrelaxation parameter for each grid resolution can be obtained, which are shown in Table 3.

Grid points	32×32	96×96	160×160	224×224
Optimal ω	1.6612	1.5584	1.5166	1.5021

Table 3 Optimal values of overrelaxation parameter ω for each grid resolution

The optimal values of overrelaxation parameter are around 1.5 and 1.6, and the optimal overrelaxation parameters decrease as the grid size decreases (smaller optimal ω for higher resolutions).

3.5 Effects of Grid Resolution on CPU Time

As the number of grid points increases, the CPU time also increases, and the relation between them satisfies:

$$T \propto N \sim M^{\frac{1}{2}}$$

Where T represents the CPU time and $M = N^2$ represents the number of grid points. Calculate the CPU time of the solving process, and plot the CPU time versus the number of grid points, shown in Figure 8.

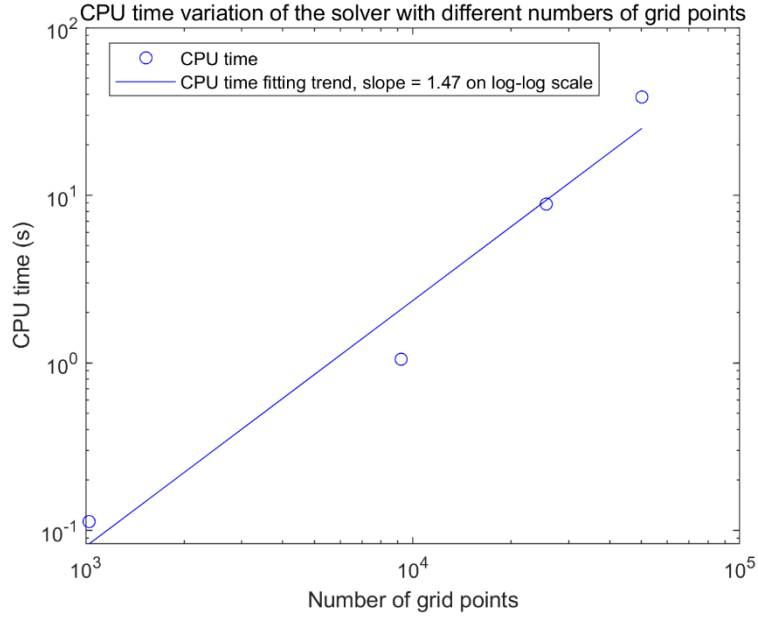


Figure 8 The CPU time variation trend of solving processes versus the number of grid points

The slope of the CPU time variation trend on log-log scale is 1.47, very close to 1.5, which indicates that the CPU time is proportional to $M^{\frac{1}{2}}$. The variation trend is as expected.

To verify the accuracy of the solution, I first run the simulation at a fine grid where $\Delta = \Delta_x = \Delta_y = 1/480$. I also select some coarser resolutions where $\Delta = [1/160, 1/154, 1/140, 1/128, 1/120, 1/96]$ to calculate the numerical solutions, and compare the solutions from coarse grids with the solution from the finest grid points. Calculate the root mean square errors of them. The root mean square errors and the variation trend of the errors versus grid sizes are shown in Figure 9.

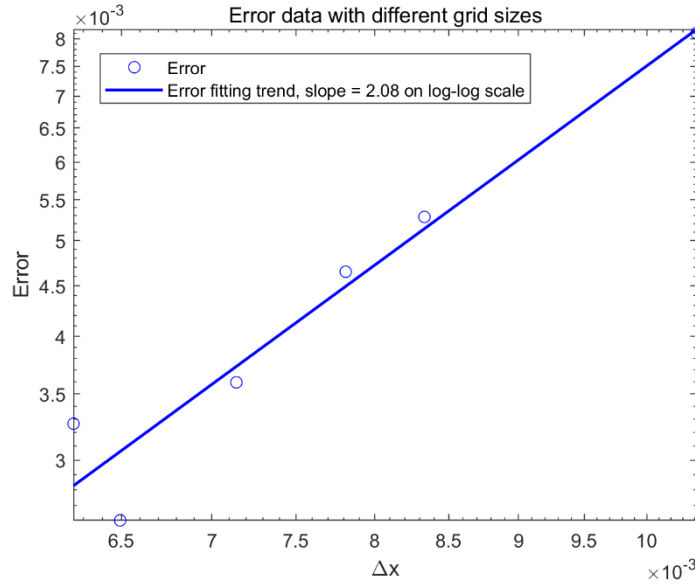


Figure 9 The root mean square errors and the variation trend versus grid sizes

The slope of the root mean square error variation trend on log-log scale is close to 2, which means $Error \sim O(\Delta x^2)$. The solutions exhibit the second-order accuracy as expected.

4. Solver Implementation for Four-circle Case

Implement the solver to a different configuration with the same outer boundary condition but with four circular inner boundaries where $u = 1$, as shown in Figure 10, and compare the results between two configurations.

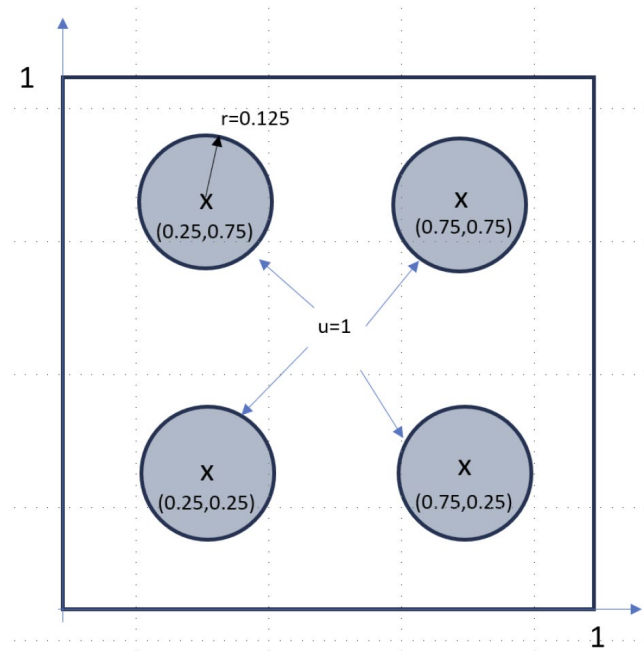


Figure 10 Configuration of the same computational domain and outer boundary with four circular

inner boundaries where $u = 1$

4.1 Iteration Process

The comparisons of the convergence performance of the point SOR Gauss-Seidel solver at the new configuration and the previous configuration for different grid resolutions are shown in Figure 11.

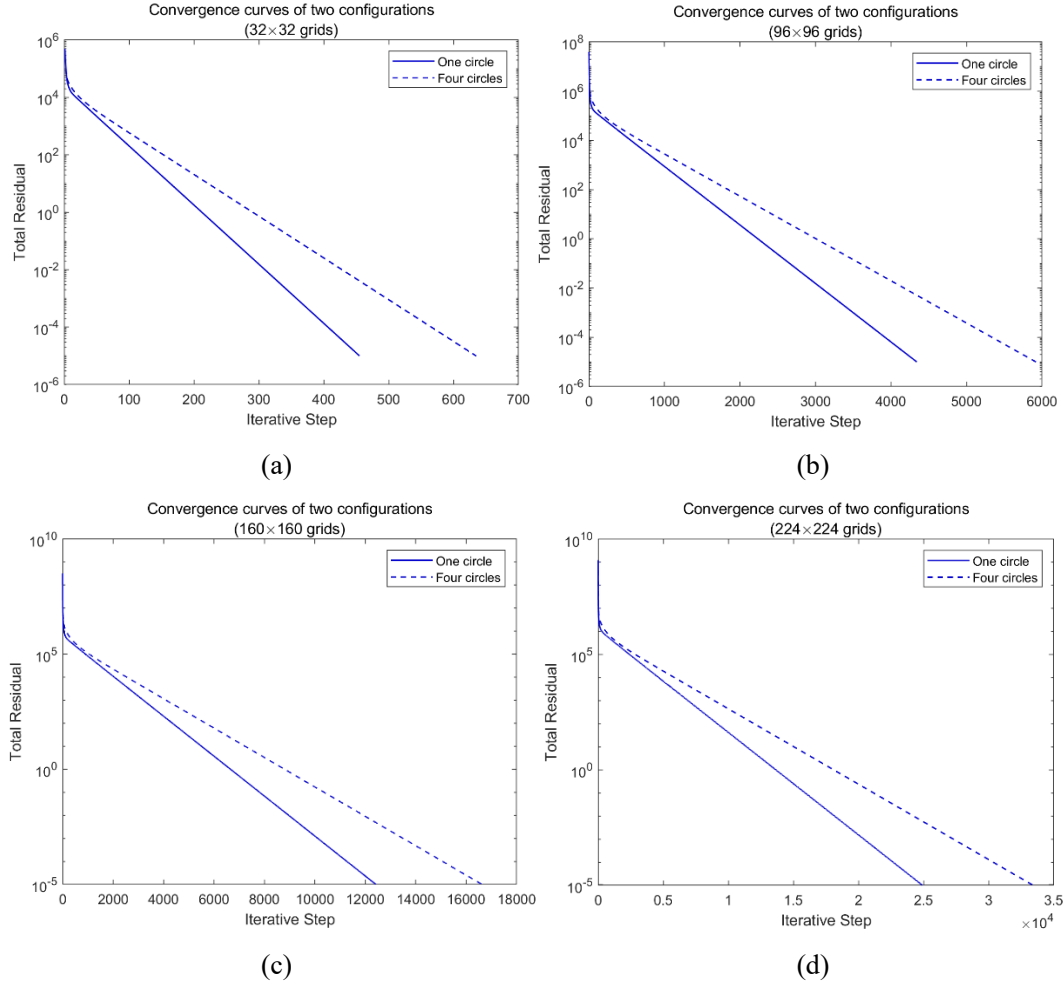


Figure 11 The total residual versus iteration index over the convergence process of the point SOR Gauss-Seidel scheme at two configurations for each grid resolution: (a) 32×32 grid, (b) 96×96 grid, (c) 160×160 grid, (d) 224×224 grid

The convergence rates of the new configuration are slower than those of the previous configuration. This result is reasonable since at the second configuration, the inner boundary condition is more complex and harder to resolve. This can be also reflected in the CPU time.

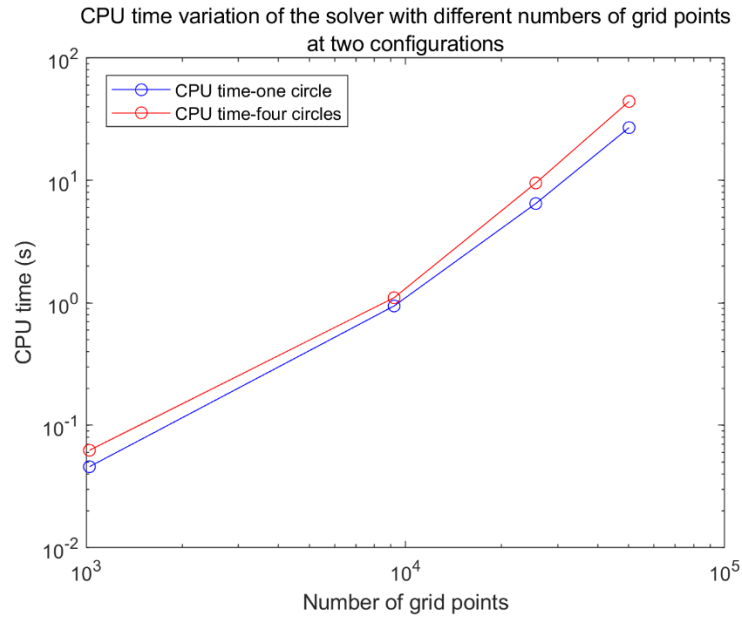
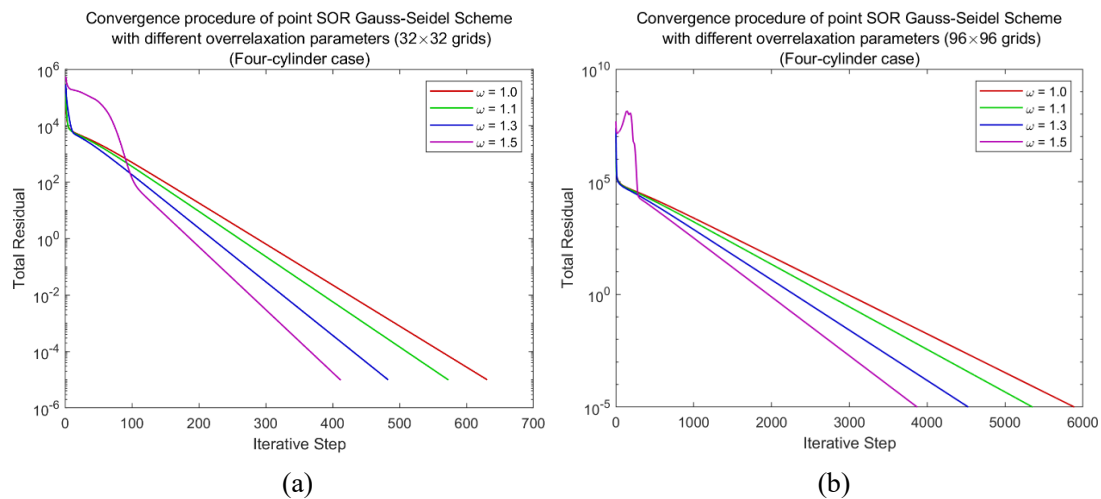


Figure 12 The CPU time variation versus the number of grid points at two configurations

Correspondingly, the CPU time of solving the second configuration is slightly longer than that of solving the first configuration due to the more complicated inner boundary condition.

4.2 Effects of Overrelaxation Parameter

For the second configuration, the convergence can also be sped up by changing the overrelaxation parameter, as shown in Figure 13. Similarly, in a proper range of the value, the larger the overrelaxation parameter, the faster the convergence for each grid resolution case.



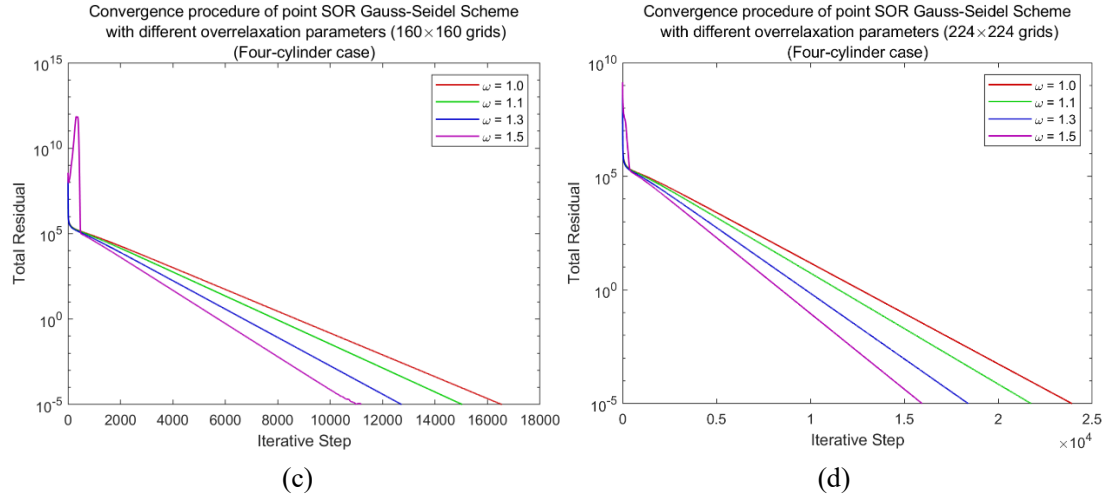
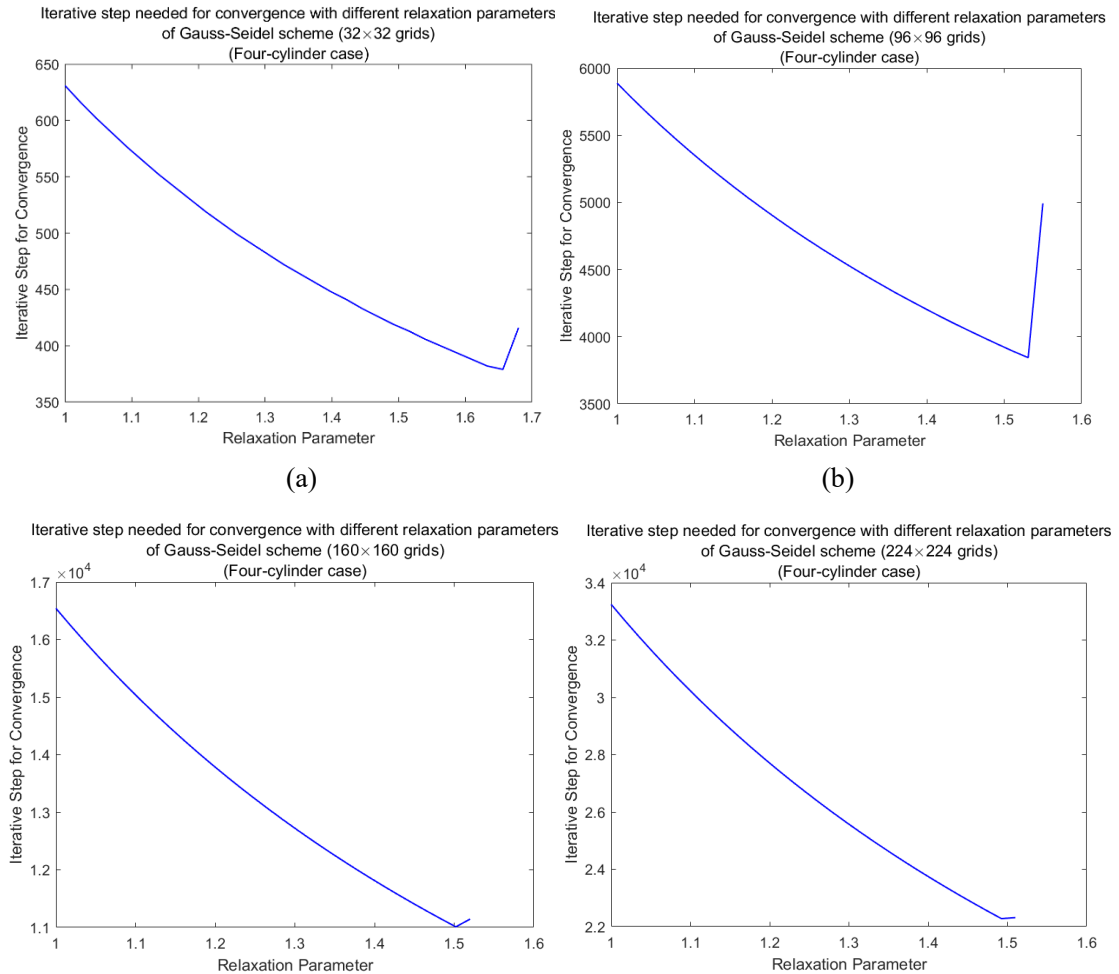


Figure 13 The convergence curves with different overrelaxation parameters at the second configuration for each grid resolution: (a) 32×32 grid, (b) 96×96 grid, (c) 160×160 grid, (d) 224×224 grid

The optimal values of the relaxation parameter can be found in the same way. The curves of iterative steps needed to converge versus different relaxation parameters for each grid resolution at the second configuration are shown in Figure 14, and the optimal values are shown in Table 4.



(c)

(d)

Figure 14 The iterative steps needed to converge with different relaxation parameters at the second configuration for each grid resolution: (a) 32×32 grid, (b) 96×96 grid, (c) 160×160 grid, (d) 224×224 grid

Grid points	32×32	96×96	160×160	224×224
Optimal ω	1.6566	1.5310	1.5021	1.4924

Table 4 Optimal values of overrelaxation parameter ω at the second configuration for each grid resolution

The optimal values of overrelaxation parameter are around 1.5 and 1.6, and the optimal overrelaxation parameters decrease as the grid sizes decrease, the same as the first configuration, while the values of the optimal overrelaxation parameter are slightly smaller than those of the first configuration.

4.3 Effects of Grid Resolution on CPU Time

Similar with the first configuration, plot the CPU time versus the number of grid points to capture the CPU time increasing trend with the increasing grid numbers, as shown in Figure 15.

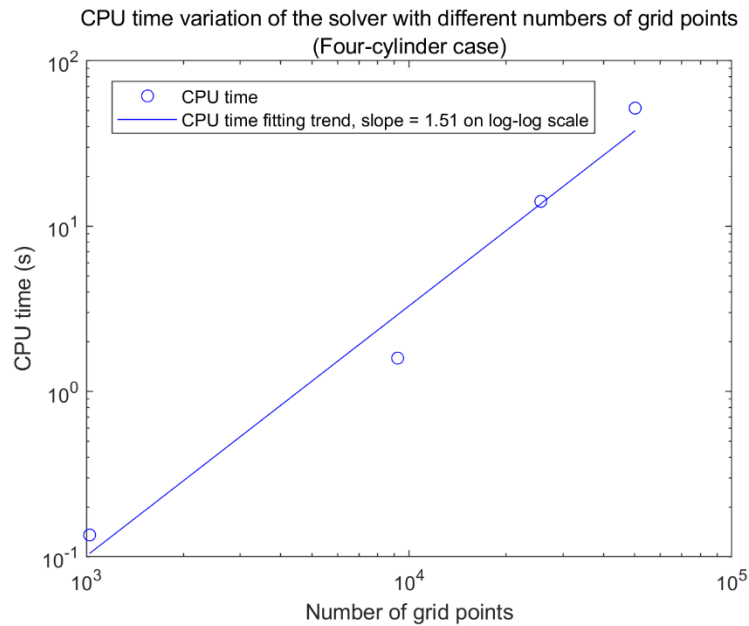


Figure 15 The CPU time variation trend versus the number of grid points at the second configuration

The slope of the CPU time variation trend on log-log scale is 1.51, indicating that the CPU time is also proportional to $M^{\frac{1}{2}}$ at the second configuration as expected. Note that the CPU time can also be influenced by the computer performance and operating

conditions in real-time.

4.4 Results Verification

Similarly, I calculate root mean square errors to verify the accuracy of the solutions at the second configuration. First the simulation is run at a fine grid where $\Delta = \Delta_x = \Delta_y = 1/480$, and then I select several grid sizes where $\Delta = [1/224, 1/180, 1/172, 1/160, 1/154, 1/96]$ to calculate numerical solutions at coarser resolutions. The solutions from coarser grids are compared to the solution from the fine grids. The errors and the variation trend are shown in Figure 16.

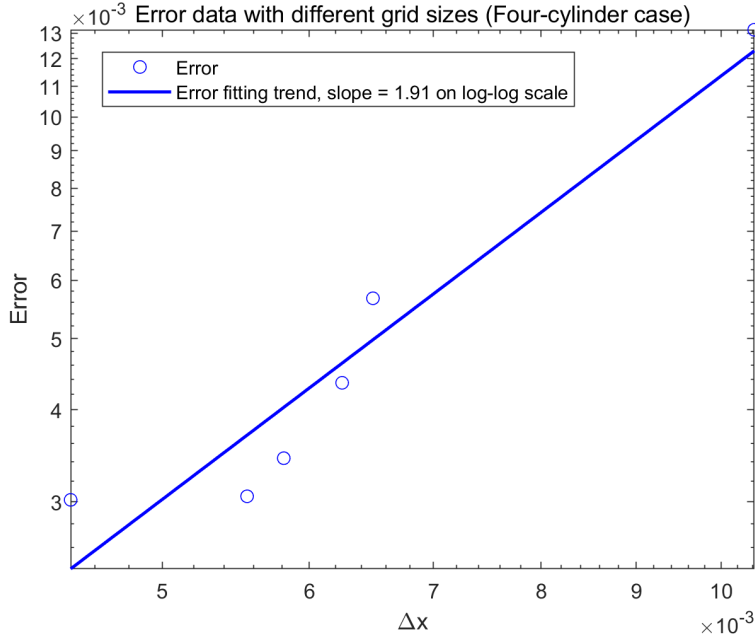


Figure 16 The root mean square errors and the variation trend versus grid sizes at the second configuration

The slope of the root mean square error variation trend on log-log scale is 1.91, close to 2, which means $Error \sim O(\Delta x^2)$. The solutions at the second configuration also exhibit the second-order accuracy as what we expect.

4.5 Implementation of Line-SOR

Implement the Line-SOR Gauss-Seidel scheme to solve the second configuration. The contours are shown in Figure 17. Note that I also assume the four circles have constant values of 1 in this case because of the inner boundary condition.

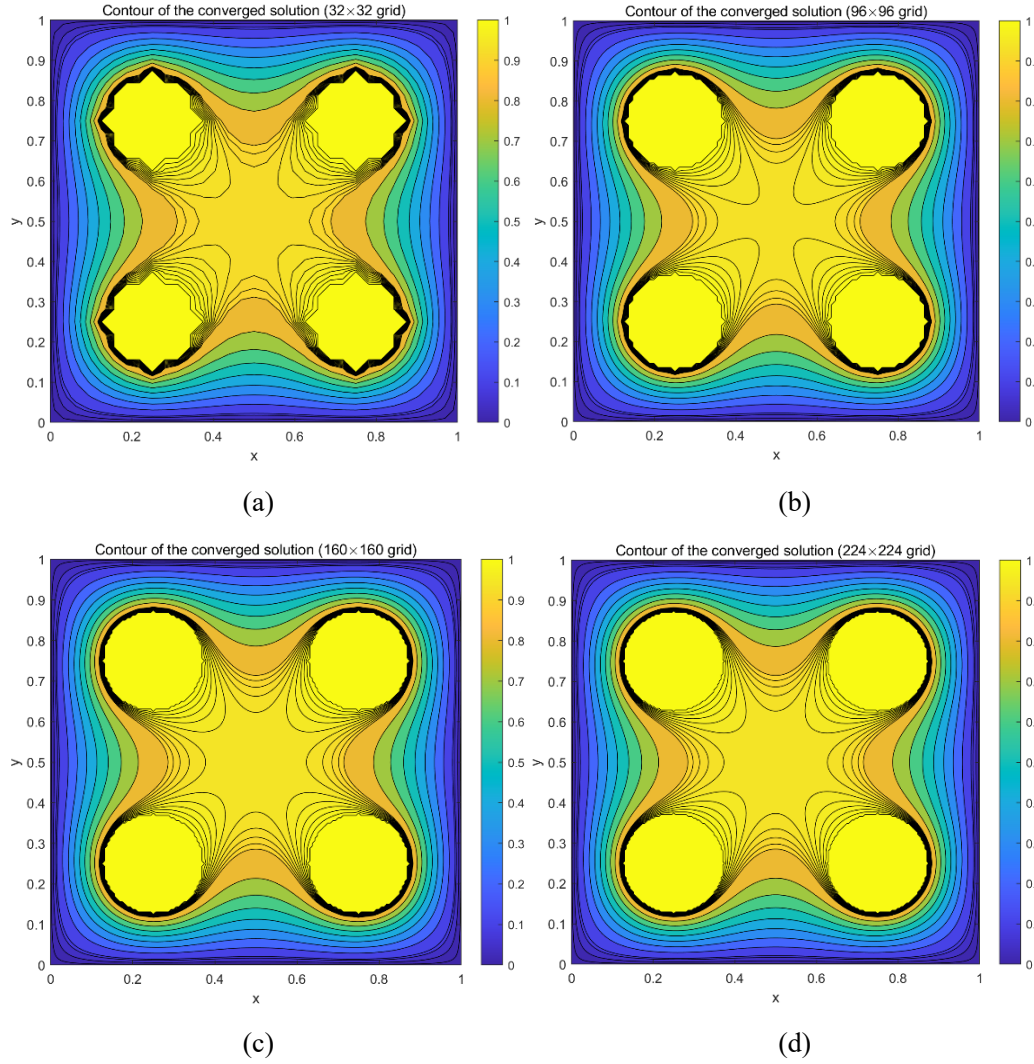


Figure 17 The contours of the converged solutions at the second configuration using Line-SOR scheme for each grid resolution: (a) 32×32 grid, (b) 96×96 grid, (c) 160×160 grid, (d) 224×224 grid

Figure 17 illustrates the contours of the solutions. The solution u decreases gradually from the four inner boundaries to the outer boundary. In addition, the gradient of the contours is steeper near the inner boundaries and gentler near the outer boundary. Higher resolutions yield more refined results. From these plots, the treatment of the inner circular boundaries of 224×224 grids are much better than that of 32×32 grids. The 32×32 grids case produces a more rudimentary operation of the inner circular boundaries, exhibiting an irregular, stair-shape appearance, whereas 224×224 grids case displays smoother curves that are closer to circles.

The convergence processes of Line-SOR scheme compared with point SOR scheme for each grid resolution at the second configuration are shown in Figure 18.

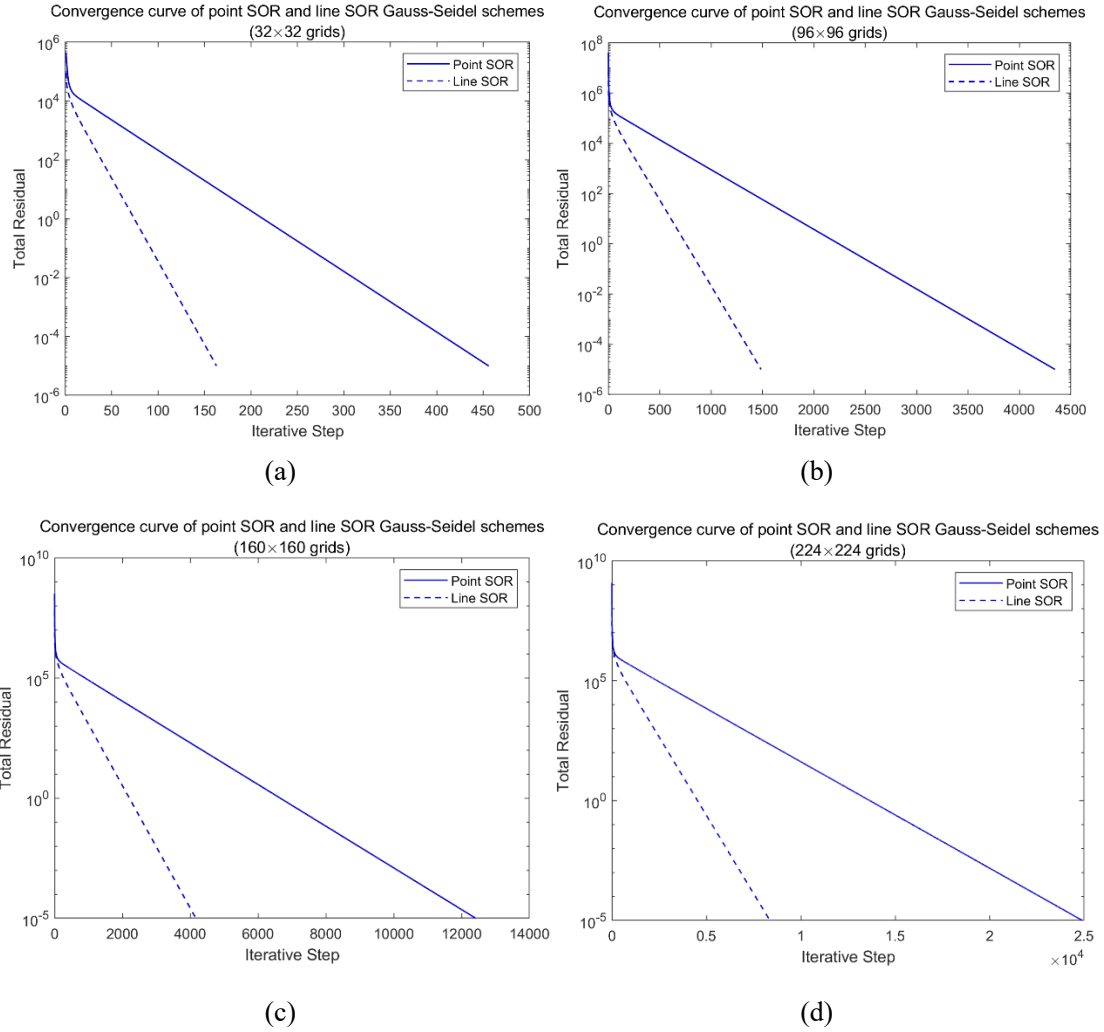


Figure 18 The total residual versus iteration index over the convergence process of the point SOR and Line-SOR Gauss-Seidel schemes at the second configuration for each grid resolution: (a) 32×32 grid, (b) 96×96 grid, (c) 160×160 grid, (d) 224×224 grid

By implementing the Line-SOR method, the u is solved line by line, instead of point by point as the point SOR scheme. Therefore, the Line-SOR results display faster convergence rates than the point SOR results, which can be observed in Figure 18.

5. The Potential Flow past the 4-cylinder Configuration

Now if I modify solver to compute the potential flow past the 4-cylinder configuration and implement the free stream conditions at the outer boundary (where the Neumann boundary condition is used), the streamlines of the flow passing the four cylinders are shown below. The resolution of 224×224 grids is used to maintain enough accuracy of the solution. First, I run this simulation in the domain shown in Figure 10 without changing the domain size. The streamlines are shown in Figure 19.

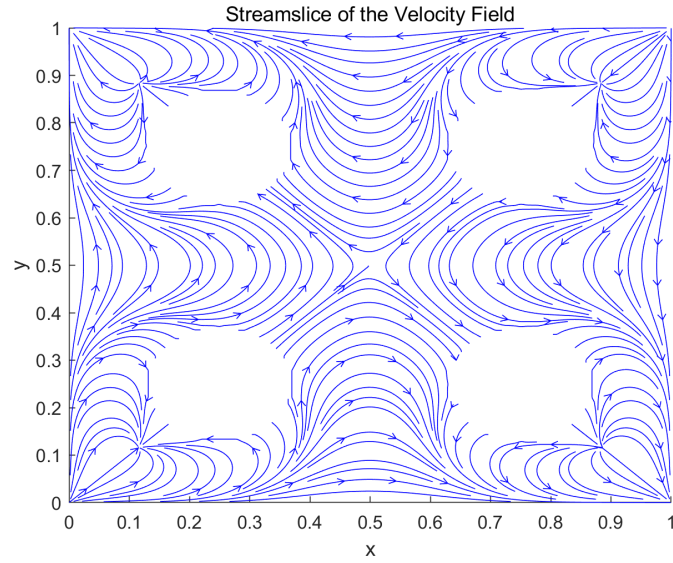


Figure 19 The streamlines of the potential flow past four-cylinder configuration in the original domain

Then, enlarge the flow domain to 9 times the original one. The corresponding streamlines are shown in Figure 20.

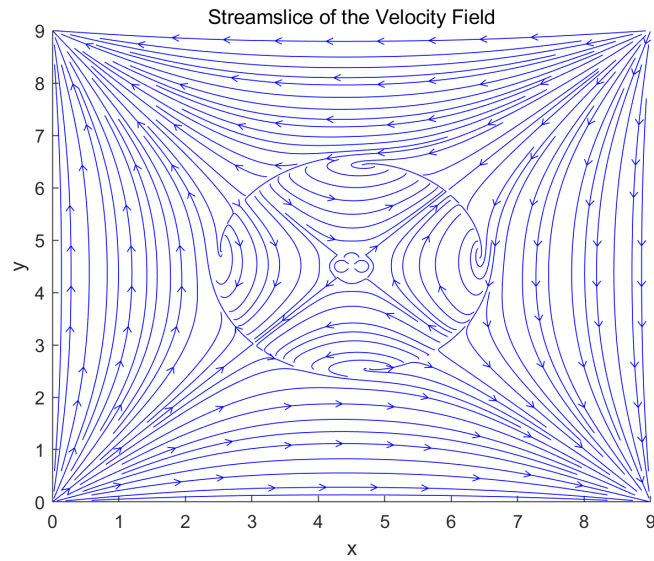


Figure 20 The streamlines of the potential flow past four-cylinder configuration in the 9 times the original domain

6. Conclusion

In summary, the 2-D Laplace equation is solved using the point Successive Over-Relaxation (SOR) Gauss-Seidel scheme within various grid resolutions. The study focuses on the equation applied to a specific domain with defined inner and outer boundary conditions. Key findings include that higher resolution grids need more iterative steps for convergence, but they yield more detailed and accurate results. A theoretical case with an altered boundary condition confirms the solver's correctness. The report also explores the influence of overrelaxation parameters on the convergence

speed, identifying optimal values for different grid sizes. The relationship between CPU time and grid size is also studied, indicating a proportional increase in computational time with grid size. Additionally, the study compares the performance of the point SOR scheme in a different configuration with multiple inner boundaries, and verify the accuracy of the solver on this configuration. The report also assesses the Line-SOR Gauss-Seidel scheme, which, despite faster convergence, demands more CPU time due to the complexities involved in solving tri-diagonal matrix equations. Further improvements for these schemes could be achieved through more extensive analytical analysis and additional numerical experiments.

7. Appendix

Source MATLAB code:

```
%% Final Project
clear;clc;

%% Plot residual
omega = 1.0;
N=32;
[c_inner32,u32_init]=initial(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
N=96;
[c_inner96,u96_init]=initial(N);
[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
N=160;
[c_inner160,u160_init]=initial(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);
N=224;
[c_inner224,u224_init]=initial(N);
[u224, rtot224] = Gauss_Seidel(c_inner224,u224_init,N,omega);
figure;
semilogy(rtot32, 'Color', [0.8, 0, 0], 'linewidth', 1);
hold on
semilogy(rtot96, 'Color', [0, 0, 0.8], 'linewidth', 1);
hold on
semilogy(rtot160, 'Color', [0, 0.8, 0], 'linewidth', 1);
hold on
semilogy(rtot224, 'Color', [0.7, 0, 0.7], 'linewidth', 1);
legend('Gauss-Seidel scheme (32\times32 grid)', 'Gauss-Seidel scheme (96\times96 grid)', 'Gauss-Seidel scheme (160\times160 grid)', 'Gauss-Seidel scheme (224\times224 grid)')
title('Total Residual versus interative index of', 'Gauss-Seidel scheme with different resolutions')
```

```

xlabel('Iterative Step')
ylabel('Total Residual')

%% Contour
% 32 grids
L=1.0;
N=32;
x = linspace(0, L, N+1);
[X, Y] = meshgrid(x, x);
figure('Position',[100, 100, 600, 500]);
contourf(X,Y,u32,'ShowText','on',"LabelFormat","%0.1f")
colorbar;
xlabel('x')
ylabel('y')
title('Contour of the converged solution (32\times32 grid)')

% 96 grids
L=1.0;
N=96;
x = linspace(0, L, N+1);
[X, Y] = meshgrid(x, x);
figure('Position',[100, 100, 600, 500]);
contourf(X,Y,u96,10,'ShowText',true,"LabelFormat","%0.1f")
colorbar;
xlabel('x')
ylabel('y')
title('Contour of the converged solution (96\times96 grid)')

% 160 grids
L=1.0;
N=160;
x = linspace(0, L, N+1);
[X, Y] = meshgrid(x, x);
figure('Position',[100, 100, 600, 500]);
contourf(X,Y,u160,10,'ShowText',true,"LabelFormat","%0.1f")
colorbar;
xlabel('x')
ylabel('y')
title('Contour of the converged solution (160\times160 grid)')

% 224 grids
L=1.0;
N=224;
x = linspace(0, L, N+1);

```

```

[X, Y] = meshgrid(x, x);
figure('Position',[100, 100, 600, 500]);
contourf(X,Y,u224,10,'ShowText',true,'LabelFormat','%0.1f')
colorbar;
xlabel('x')
ylabel('y')
title('Contour of the converged solution (224\times224 grid)')

%% check correctness
[res32, err32, u_exact, u32]=find_error(32);
[res96, err96, u_exact, u96]=find_error(96);
[res160, err160, u_exact, u160]=find_error(160);
[res224, err224, u_exact, u224]=find_error(224);

%% Overrelaxation
N=32;
[c_inner32,u32_init]=initial(N);
omega = 1.0;
[u32_w10, rtot32_w10] = Gauss_Seidel(c_inner32,u32_init,N,omega);
omega = 1.1;
[u32_w11, rtot32_w11] = Gauss_Seidel(c_inner32,u32_init,N,omega);
omega = 1.3;
[u32_w13, rtot32_w13] = Gauss_Seidel(c_inner32,u32_init,N,omega);
omega = 1.5;
[u32_w15, rtot32_w15] = Gauss_Seidel(c_inner32,u32_init,N,omega);

figure;
semilogy(rtot32_w10,'Color',[0.8, 0, 0], 'linewidth', 1);
hold on
semilogy(rtot32_w11, 'Color',[0, 0.8, 0], 'linewidth', 1);
hold on
semilogy(rtot32_w13, 'Color',[0, 0, 0.8], 'linewidth', 1);
hold on
semilogy(rtot32_w15, 'Color',[0.7, 0, 0.7], 'linewidth', 1);
legend('\omega = 1.0', '\omega = 1.1', '\omega = 1.3', '\omega = 1.5')
title({'Convergence procedure of point SOR Gauss-Seidel Scheme' ...
      ' with different overrelaxation parameters (32\times32 grids)'});
xlabel('Iterative Step')
ylabel('Total Residual')

%% Find optimal
omega = linspace(1.0,1.78,30);
iter=[];
for i = 1:length(omega)

```

```

        [u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega(i));
        iter=[iter,length(rtot32)];
    end
    figure;
    plot(omega,iter,'b-','linewidth',1)
    title({'Iterative step needed for convergence with different relaxation'
    parameters', ...
        'of Gauss-Seidel scheme (32\times32 grids)'});
    xlabel('Relaxation Parameter')
    ylabel('Iterative Step for Convergence')
    [minvalue,i]=min(iter);
    omega_opt=omega(i)

%% Calculate running time
tic
N=32;
omega=1;
[c_inner32,u32_init]=initial(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
elapsedTime32 = toc;

tic
N=96;
omega=1;
[c_inner96,u96_init]=initial(N);
[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
elapsedTime96 = toc;

tic
N=160;
omega=1;
[c_inner160,u160_init]=initial(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);
elapsedTime160 = toc;

tic
N=224;
omega=1;
[c_inner224,u224_init]=initial(N);
[u224, rtot224] = Gauss_Seidel(c_inner224,u224_init,N,omega);
elapsedTime224 = toc;

grid_points=[32^2 96^2 160^2 224^2];
elapsedTime=[elapsedTime32 elapsedTime96 elapsedTime160 elapsedTime224];

```

```

logDx = log10(grid_points);
logDy = log10(elapsedTime);
p = polyfit(logDx, logDy, 1);
fitLogy = polyval(p, logDx);
figure(1)
loglog(grid_points, elapsedTime, 'bo');
hold on;
loglog(grid_points, 10.^fitLogy, 'b-');
hold off;
xlabel('Number of grid points');
ylabel('CPU time (s)');
title('CPU time variation of the solver with different numbers of grid
points')
legend('CPU time', sprintf('CPU time fitting trend, slope = %.2f on log-log
scale', p(1)))

```

```

%% Accuracy

```

```

omega = 1.0;
N=96;
[c_inner96,u96_init]=initial(N);
[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
N=120;
[c_inner120,u120_init]=initial(N);
[u120, rtot120] = Gauss_Seidel(c_inner120,u120_init,N,omega);
N=128;
[c_inner128,u128_init]=initial(N);
[u128, rtot128] = Gauss_Seidel(c_inner128,u128_init,N,omega);
N=140;
[c_inner140,u140_init]=initial(N);
[u140, rtot140] = Gauss_Seidel(c_inner140,u140_init,N,omega);
N=154;
[c_inner154,u154_init]=initial(N);
[u154, rtot154] = Gauss_Seidel(c_inner154,u154_init,N,omega);
N=160;
[c_inner160,u160_init]=initial(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);
N=480;
[c_inner480,u480_init]=initial(N);
[u480, rtot480] = Gauss_Seidel(c_inner480,u480_init,N,omega);

[XFine, YFine] = meshgrid(linspace(0, 1, 481), linspace(0, 1, 481));

[X96, Y96] = meshgrid(linspace(0, 1, 97), linspace(0, 1, 97));
inter96 = interp2(XFine, YFine, u480, X96, Y96);

```

```

errorMatrix = abs(u96 - inter96);
rmsError96 = sqrt(mean(errorMatrix.^2));

[X120, Y120] = meshgrid(linspace(0, 1, 121), linspace(0, 1, 121));
inter120 = interp2(XFine, YFine, u480, X120, Y120);
errorMatrix = abs(u120 - inter120);
rmsError120 = sqrt(mean(errorMatrix.^2));

[X128, Y128] = meshgrid(linspace(0, 1, 129), linspace(0, 1, 129));
inter128 = interp2(XFine, YFine, u480, X128, Y128);
errorMatrix = abs(u128 - inter128);
rmsError128 = sqrt(mean(errorMatrix.^2));

[X140, Y140] = meshgrid(linspace(0, 1, 141), linspace(0, 1, 141));
inter140 = interp2(XFine, YFine, u480, X140, Y140);
errorMatrix = abs(u140 - inter140);
rmsError140 = sqrt(mean(errorMatrix.^2));

[X154, Y154] = meshgrid(linspace(0, 1, 155), linspace(0, 1, 155));
inter154 = interp2(XFine, YFine, u480, X154, Y154);
errorMatrix = abs(u154 - inter154);
rmsError154 = sqrt(mean(errorMatrix.^2));

[X160, Y160] = meshgrid(linspace(0, 1, 161), linspace(0, 1, 161));
inter160 = interp2(XFine, YFine, u480, X160, Y160);
errorMatrix = abs(u160 - inter160);
rmsError160 = sqrt(mean(errorMatrix.^2));

dx = [1/160 1/154 1/140 1/128 1/120 1/96];
er = [rmsError160 rmsError154 rmsError140 rmsError128 rmsError120
rmsError96];

logDx = log10(dx);
logEr = log10(er);
p1 = polyfit(logDx, logEr, 1);
fitLogEr = polyval(p1, logDx);
figure;
loglog(dx, er, 'bo');
hold on;
loglog(dx, 10.^fitLogEr, 'b-', 'linewidth',1.5);
slope = p1(1);

xlabel('\Delta x');
ylabel('Error');

```

```

title('Error data with different grid sizes')
legend('Error', sprintf('Error fitting trend, slope = %.2f on log-log
scale', slope))

%% Four-cylinder case
% Compare convergence
omega = 1.0;
N=32;
[c_inner32, u32_init] = initial_4c(N);
[u32_4c, rtot32_4c] = Gauss_Seidel(c_inner32,u32_init,N,omega);
[c_inner32,u32_init]=initial(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
figure;
semilogy(rtot32, 'Color', [0, 0, 0.8], 'linewidth', 1);
hold on
semilogy(rtot32_4c, '--', 'Color', [0, 0, 0.8], 'linewidth', 1);
legend('One circle','Four circles')
title('Convergence curves of two configurations', '(32\times32 grids)')
xlabel('Iterative Step')
ylabel('Total Residual')

%% Compare running time - Four-cylinder
omega=1;
tic
N=32;
[c_inner32,u32_init]=initial(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
elapsedTime32 = toc;

tic
N=96;
[c_inner96,u96_init]=initial(N);
[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
elapsedTime96 = toc;

tic
N=160;
[c_inner160,u160_init]=initial(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);
elapsedTime160 = toc;

tic
N=224;
[c_inner224,u224_init]=initial(N);

```



```

[u224, rtot224] = Gauss_Seidel(c_inner224,u224_init,N,omega);
elapsedTime224 = toc;

grid_points=[32^2 96^2 160^2 224^2];
elapsedTime=[elapsedTime32 elapsedTime96 elapsedTime160 elapsedTime224];
figure;
loglog(grid_points, elapsedTime, 'bo-');

tic
N=32;
[c_inner32,u32_init]=initial_4c(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
elapsedTime32 = toc;

tic
N=96;
[c_inner96,u96_init]=initial_4c(N);
[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
elapsedTime96 = toc;

tic
N=160;
[c_inner160,u160_init]=initial_4c(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);
elapsedTime160 = toc;

tic
N=224;
[c_inner224,u224_init]=initial_4c(N);
[u224, rtot224] = Gauss_Seidel(c_inner224,u224_init,N,omega);
elapsedTime224 = toc;

elapsedTime=[elapsedTime32 elapsedTime96 elapsedTime160 elapsedTime224];
hold on
loglog(grid_points, elapsedTime, 'ro-');

xlabel('Number of grid points');
ylabel('CPU time (s)');
title({'CPU time variation of the solver with different numbers of grid points', 'at two configurations'})
legend('CPU time-one circle', 'CPU time-four circles')

%% Overrelaxation - Four-cylinder
N=32;

```

```

[c_inner32,u32_init]=initial_4c(N);
omega = 1.0;
[u32_w10, rtot32_w10] = Gauss_Seidel(c_inner32,u32_init,N,omega);
omega = 1.1;
[u32_w11, rtot32_w11] = Gauss_Seidel(c_inner32,u32_init,N,omega);
omega = 1.3;
[u32_w13, rtot32_w13] = Gauss_Seidel(c_inner32,u32_init,N,omega);
omega = 1.5;
[u32_w15, rtot32_w15] = Gauss_Seidel(c_inner32,u32_init,N,omega);

figure;
semilogy(rtot32_w10,'Color',[0.8, 0, 0], 'linewidth', 1);
hold on
semilogy(rtot32_w11, 'Color',[0, 0.8, 0], 'linewidth', 1);
hold on
semilogy(rtot32_w13, 'Color',[0, 0, 0.8], 'linewidth', 1);
hold on
semilogy(rtot32_w15, 'Color',[0.7, 0, 0.7], 'linewidth', 1);
legend('\omega = 1.0', '\omega = 1.1', '\omega = 1.3', '\omega = 1.5')
title({'Convergence procedure of point SOR Gauss-Seidel Scheme' ...
      ' with different overrelaxation parameters (32\times32 grids)' ...
      '(Four-cylinder case)'});
xlabel('Iterative Step')
ylabel('Total Residual')

%% Find optimal - Four-cylinder
omega = linspace(1.0,1.68,30);
N=32;
[c_inner32,u32_init]=initial_4c(N);
iter=[];
for i = 1:length(omega)
    [u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega(i));
    iter=[iter,length(rtot32)];
end
figure;
plot(omega,iter,'b-','linewidth',1)
title({'Iterative step needed for convergence with different relaxation
parameters', ...
      'of Gauss-Seidel scheme (32\times32 grids)' ...
      '(Four-cylinder case)'}, 'fontweight', 'normal');
xlabel('Relaxation Parameter')
ylabel('Iterative Step for Convergence')
[minvalue,i]=min(iter);
omega_opt=omega(i)

```

```

%% Calculate running time - Four-cylinder
tic
N=32;
omega=1;
[c_inner32,u32_init]=initial(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
elapsedTime32 = toc;

tic
N=96;
omega=1;
[c_inner96,u96_init]=initial(N);
[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
elapsedTime96 = toc;

tic
N=160;
omega=1;
[c_inner160,u160_init]=initial(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);
elapsedTime160 = toc;

tic
N=224;
omega=1;
[c_inner224,u224_init]=initial(N);
[u224, rtot224] = Gauss_Seidel(c_inner224,u224_init,N,omega);
elapsedTime224 = toc;

grid_points=[32^2 96^2 160^2 224^2];
elapsedTime=[elapsedTime32 elapsedTime96 elapsedTime160 elapsedTime224];
figure;
loglog(grid_points, elapsedTime, 'bo-');

tic
N=32;
[c_inner32,u32_init]=initial_4c(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
elapsedTime32 = toc;

tic
N=96;
[c_inner96,u96_init]=initial_4c(N);

```

```

[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
elapsedTime96 = toc;

tic
N=160;
[c_inner160,u160_init]=initial_4c(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);
elapsedTime160 = toc;

tic
N=224;
[c_inner224,u224_init]=initial_4c(N);
[u224, rtot224] = Gauss_Seidel(c_inner224,u224_init,N,omega);
elapsedTime224 = toc;

elapsedTime=[elapsedTime32 elapsedTime96 elapsedTime160 elapsedTime224];
hold on
loglog(grid_points, elapsedTime, 'ro-');

xlabel('Number of grid points');
ylabel('CPU time (s)');
title({'CPU time variation of the solver with different numbers of grid points' ...
      'at two configurations'})
legend('CPU time-one circle', 'CPU time-four circles')

%% Accuracy - Four-cylinder case
omega = 1.0;
N=96;
[c_inner96,u96_init]=initial_4c(N);
[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
N=154;
[c_inner154,u154_init]=initial_4c(N);
[u154, rtot154] = Gauss_Seidel(c_inner154,u154_init,N,omega);
N=160;
[c_inner160,u160_init]=initial_4c(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);
N=172;
[c_inner172,u172_init]=initial_4c(N);
[u172, rtot172] = Gauss_Seidel(c_inner172,u172_init,N,omega);
N=180;
[c_inner180,u180_init]=initial_4c(N);
[u180, rtot180] = Gauss_Seidel(c_inner180,u180_init,N,omega);
N=224;

```

```

[c_inner224,u224_init]=initial_4c(N);
[u224, rtot224] = Gauss_Seidel(c_inner224,u224_init,N,omega);
N=480;
[c_inner480,u480_init]=initial_4c(N);
[u480, rtot480] = Gauss_Seidel(c_inner480,u480_init,N,omega);

[XFine, YFine] = meshgrid(linspace(0, 1, 481), linspace(0, 1, 481));

[X96, Y96] = meshgrid(linspace(0, 1, 97), linspace(0, 1, 97));
inter96 = interp2(XFine, YFine, u480, X96, Y96);
errorMatrix = abs(u96 - inter96);
rmsError96 = sqrt(mean(errorMatrix.^2));

[X154, Y154] = meshgrid(linspace(0, 1, 155), linspace(0, 1, 155));
inter154 = interp2(XFine, YFine, u480, X154, Y154);
errorMatrix = abs(u154 - inter154);
rmsError154 = sqrt(mean(errorMatrix.^2));

[X160, Y160] = meshgrid(linspace(0, 1, 161), linspace(0, 1, 161));
inter160 = interp2(XFine, YFine, u480, X160, Y160);
errorMatrix = abs(u160 - inter160);
rmsError160 = sqrt(mean(errorMatrix.^2));

[X172, Y172] = meshgrid(linspace(0, 1, 173), linspace(0, 1, 173));
inter172 = interp2(XFine, YFine, u480, X172, Y172);
errorMatrix = abs(u172 - inter172);
rmsError172 = sqrt(mean(errorMatrix.^2));

[X180, Y180] = meshgrid(linspace(0, 1, 181), linspace(0, 1, 181));
inter180 = interp2(XFine, YFine, u480, X180, Y180);
errorMatrix = abs(u180 - inter180);
rmsError180 = sqrt(mean(errorMatrix.^2));

[X224, Y224] = meshgrid(linspace(0, 1, 225), linspace(0, 1, 225));
inter224 = interp2(XFine, YFine, u480, X224, Y224);
errorMatrix = abs(u224 - inter224);
rmsError224 = sqrt(mean(errorMatrix.^2));

dx = [1/224 1/180 1/172 1/160 1/154 1/96];
er = [rmsError224 rmsError180 rmsError172 rmsError160 rmsError154
rmsError96];

logDx = log10(dx);
logEr = log10(er);

```

```

p1 = polyfit(logDx, logEr, 1);
fitLogEr = polyval(p1, logDx);
figure;
loglog(dx, er, 'bo');
hold on;
loglog(dx, 10.^fitLogEr, 'b-', 'linewidth',1.5);
slope = p1(1);

xlabel('\Delta x');
ylabel('Error');
title('Error data with different grid sizes (Four-cylinder case)')
legend('Error', sprintf('Error fitting trend, slope = %.2f on log-log scale', slope))

%% Line SOR
% Convergence
omega = 1.0;
N=32;
[u32sor, rtot32sor] = line_sor(N,omega);
[c_inner32,u32_init]=initial(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
figure;
semilogy(rtot32, 'Color', [0, 0, 0.8], 'linewidth', 1);
hold on
semilogy(rtot32sor, '--', 'Color', [0, 0, 0.8], 'linewidth', 1);
legend('Point SOR','Line SOR')
title('Convergence curve of point SOR and line SOR Gauss-Seidel schemes',
'(32\times32 grids)')
xlabel('Iterative Step')
ylabel('Total Residual')

%% Contour - Line SOR
L=1.0;
N=32;
x = linspace(0, L, N+1);
[X, Y] = meshgrid(x, x);
figure('Position',[100, 100, 600, 500]);
contourf(X,Y,u32sor,[0.0 0.01 0.04 0.08 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99
1.0], 'ShowText', 'off', 'LabelFormat', '%0.1f')
colorbar;
xlabel('x')
ylabel('y')
title('Contour of the converged solution (32\times32 grid)')

```

```

%% Calculate running time - Line SOR
omega=1;
tic
N=32;
[u32, rtot32] = line_sor(N,omega);
elapsedTime32 = toc;

tic
N=96;
[u96, rtot96] = line_sor(N,omega);
elapsedTime96 = toc;

tic
N=160;
[u160, rtot160] = line_sor(N,omega);
elapsedTime160 = toc;

tic
N=224;
[u224, rtot224] = line_sor(N,omega);
elapsedTime224 = toc;

grid_points=[32^2 96^2 160^2 224^2];
elapsedTime=[elapsedTime32 elapsedTime96 elapsedTime160 elapsedTime224];
figure;
loglog(grid_points, elapsedTime, 'ro-');

tic
N=32;
[c_inner32,u32_init]=initial_4c(N);
[u32, rtot32] = Gauss_Seidel(c_inner32,u32_init,N,omega);
elapsedTime32 = toc;

tic
N=96;
[c_inner96,u96_init]=initial_4c(N);
[u96, rtot96] = Gauss_Seidel(c_inner96,u96_init,N,omega);
elapsedTime96 = toc;

tic
N=160;
[c_inner160,u160_init]=initial_4c(N);
[u160, rtot160] = Gauss_Seidel(c_inner160,u160_init,N,omega);

```

```

elapsedTime160 = toc;

tic
N=224;
[c_inner224,u224_init]=initial_4c(N);
[u224, rtot224] = Gauss_Seidel(c_inner224,u224_init,N,omega);
elapsedTime224 = toc;

elapsedTime=[elapsedTime32 elapsedTime96 elapsedTime160 elapsedTime224];
hold on
loglog(grid_points, elapsedTime, 'bo-');

xlabel('Number of grid points');
ylabel('CPU time (s)');
title({'CPU time variation of point SOR and line SOR Gauss-Seidel schemes',
'with different numbers of grid points'})
legend('CPU time-line', 'CPU time-point')

%% Streamlines
L = 1.0;
N = 224;
omega=1;
[c_inner96,u96_init]=initial_4c(N);
[u96, rtot96] = Gauss_Seidel_nmbc(c_inner96,u96_init,N,omega);

x = linspace(0, L, N+1);
y = linspace(0, L, N+1);
[X, Y] = meshgrid(x, y);

[v, u] = gradient(u96, x(2)-x(1), y(2)-y(1));

figure;
streamslice(X, Y, u, v, 3);
title('Streamslice of the Velocity Field');
xlabel('x');
ylabel('y');

%% Functions
function [u,rtota]=line_sor(N,omega)
[c_inner, u_init]=initial_4c(N);
u=u_init;
unew=u;
rtot=1;
rtota=[];

```



```

b = ones(N-1, 1);
dx=1/N;
dx2=dx^2;
while rtot>1e-5
rtot=0;
temp=u;

% row
for i = 2:N
    mainDiagx = (1 - c_inner(i, 2:N)) * (-4) + c_inner(i, 2:N);
    b(1:N-1) = -(1 - c_inner(i, 2:N)) .* (temp(i-1, 2:N) + temp(i+1, 2:N))
+ c_inner(i, 2:N);
    upperDiag = 1 - c_inner(i, 2:N-1);
    lowerDiag = 1 - c_inner(i, 3:N);

    triDiagMatrix_row = diag(mainDiagx) + diag(upperDiag, 1) +
diag(lowerDiag, -1);
    unew(i,2:N)=triDiagMatrix_row\b;
    temp(i,2:N)=(1-omega).*u(i,2:N)+omega.*unew(i,2:N);
    unew(i,2:N)=temp(i,2:N);
end

% column
for j = 2:N
    mainDiagx = (1 - c_inner(2:N, j)) * (-4.0) + c_inner(2:N, j);
    b(1:N-1) = -(1 - c_inner(2:N, j)) .* (temp(2:N, j-1) + temp(2:N, j+1))
+ c_inner(2:N, j);
    upperDiag = 1 - c_inner(2:N-1, j);
    lowerDiag = 1 - c_inner(3:N, j);

    triDiagMatrix_row = diag(mainDiagx) + diag(upperDiag, 1) +
diag(lowerDiag, -1);
    unew(2:N, j)=triDiagMatrix_row\b;
    temp(2:N, j)=(1-omega).*u(2:N,j)+omega.*unew(2:N,j);
    unew(2:N, j)=temp(2:N, j);
end

u=unew;
for i = 2:N
    for j = 2:N
        r(i,j)=abs( ...
            (1-c_inner(i,j))* ...

```

```

        ((u(i-1,j)-2*u(i,j)+u(i+1,j))/dx2+(u(i,j-1)-
2*u(i,j)+u(i,j+1))/dx2));
        rtot = rtot + r(i,j);
    end
end
rtota=[rtota,rtot];
end

end

function [residual, error_relative, u_exact, u]=find_error(N)
R=sqrt(2)/2;
r1=0.25;
omega = 1.0;
[c_inner,u_init]=initial(N);
[u_initial]=change_boundary(u_init,N);
[u,rtot] = Gauss_Seidel(c_inner,u_initial,N,omega);

x = linspace(-0.5, 0.5, N+1);
[X, Y] = meshgrid(x, x);
dist_squared = sqrt(X.^2 + Y.^2);
u_exact = (log(dist_squared)-log(R))/(log(r1)-log(R));

u_exact(N/2+1,N/2+1)=1;
index_c_inner = find(c_inner==0);
num_0_c_inner = length(index_c_inner)-4*N;
residual = sum(abs((1-c_inner).*(u_exact-u)))/num_0_c_inner;
error_relative=sqrt(sum((1-c_inner(2:N,2:N)).*(u_exact(2:N,2:N)-
u(2:N,2:N)).^2, 'all')/num_0_c_inner);

end

function [c_inner,u]=initial_4c(N)
L = 1.0;
r = 0.125;
cen1 = 0.25;
cen2 = 0.75;
x = linspace(0, L, N+1);
[X, Y] = meshgrid(x, x);

dist_squared1 = (X - cen1).^2 + (Y - cen2).^2;
dist_squared2 = (X - cen2).^2 + (Y - cen2).^2;
dist_squared3 = (X - cen1).^2 + (Y - cen1).^2;
dist_squared4 = (X - cen2).^2 + (Y - cen1).^2;

```

```

c_inner = zeros(N+1, N+1);
c_inner(dist_squared1 <= r^2) = 1;
c_inner(dist_squared2 <= r^2) = 1;
c_inner(dist_squared3 <= r^2) = 1;
c_inner(dist_squared4 <= r^2) = 1;

u = zeros(N+1, N+1);
randomMatrix = 2 * rand(N+1, N+1) - 1;
u(2:N,2:N) = (1 - c_inner(2:N,2:N)) .* randomMatrix(2:N,2:N) +
c_inner(2:N,2:N) * 1.0;

```

end

```

function [u] = change_boundary(u_init,N)
dx=1/N;
R=sqrt(2)/2;
r1=0.25;
u=u_init;
for i = 1:N+1
    xe=-0.5+(i-1)*dx;
    r=sqrt(xe^2+0.5^2);
    u(1,i)=(log(r)-log(R))/(log(r1)-log(R));
    u(N+1,i)=(log(r)-log(R))/(log(r1)-log(R));
    u(i,1)=(log(r)-log(R))/(log(r1)-log(R));
    u(i,N+1)=(log(r)-log(R))/(log(r1)-log(R));
end

```

end

end

```

function [c_inner,u] = initial(N)
L = 1.0;
r = 0.25;
cen = 0.5;
x = linspace(0, L, N+1);
[X, Y] = meshgrid(x, x);

dist_squared = (X - cen).^2 + (Y - cen).^2;
c_inner = ones(N+1, N+1);
c_inner(dist_squared > r^2) = 0;

u = zeros(N+1, N+1);
randomMatrix = 2 * rand(N+1, N+1) - 1;
u(2:N,2:N) = (1 - c_inner(2:N,2:N)) .* randomMatrix(2:N,2:N) +
c_inner(2:N,2:N) * 1.0;

```

end

```
function [u,rtota] = Gauss_Seidel(c_inner,u_init,N,omega)
u = u_init;
unew=u;
rtot=1;
rtota=[];
while rtot>1e-5
    rtot=0;
    temp=u;
    for i = 2:N
        for j = 2:N
            unew(i,j) = (1 - c_inner(i,j))*(1/4*(temp(i+1,j)+temp(i-1,j)+temp(i,j+1)+temp(i,j-1)))+c_inner(i,j) * 1.0;
            temp(i,j)=(1-omega)*u(i,j)+omega*unew(i,j);
            unew(i,j) = temp(i,j);
        end
    end
    u=unew;
    dx=1/N;
    for i = 2:N
        for j = 2:N
            r(i,j)=abs( ...
                (1-c_inner(i,j))* ...
                ((u(i-1,j)-2*u(i,j)+u(i+1,j))/dx^2+(u(i,j-1)-2*u(i,j)+u(i,j+1))/dx^2));
            rtot = rtot + r(i,j);
        end
    end
    rtota=[rtota,rtot];
end
end
```

```
function [u,rtota] = Gauss_Seidel_nmbc(c_inner,u_init,N,omega)
u = u_init;
unew=u;
rtot=1;
rtota=[];
L=1.0;
while rtot>1e-5
    rtot=0;
    temp=u;
    dx=L/N;
```

```

for i = 2:N
    for j = 2:N
        unew(i,j) = (1 - c_inner(i,j))*(1/4*(temp(i+1,j)+temp(i-
1,j)+temp(i,j+1)+temp(i,j-1)));
        temp(i,j)=(1-omega)*u(i,j)+omega*unew(i,j);
        unew(i,j) = temp(i,j);
    end
end

for j = 1:N+1
    u(1,j) = u(2,j)-dx
    u(N+1,j) = u(N,j)+dx;
end
for i = 1:N+1
    u(i,1) = u(i,2);
    u(i,N+1) = u(i,N);
end
if c_inner(i,j)==1
    neighbors = [];

    neighbors(end+1) = u(i-1,j);
    neighbors(end+1) = u(i+1,j);
    neighbors(end+1) = u(i,j+1);
    neighbors(end+1) = u(i,j-1);

    u(i,j) = max(neighbors);
end

u=unew;

for i = 2:N
    for j = 2:N
        r(i,j)=abs( ...
            (1-c_inner(i,j))* ...
            ((u(i-1,j)-2*u(i,j)+u(i+1,j))/dx^2+(u(i,j-1)-
2*u(i,j)+u(i,j+1))/dx^2));
        rtot = rtot + r(i,j);
    end
end
rtota=[rtota,rtot];
end
end

```